

Laboratory work: lab2

Student: FangJiahui

Student ID: 24320124

Timus Name:hduads2026_24320124

Mail: 3892551687@qq.com

Problem:1401, 2025

Screenshot from Timus:

1401:

Recent submissions

Author: [hduads2026_24320124](#) • Problem: [Gamers](#)

ID	Date	Author	Problem	Language	Judgement result	Test #	Execution time	Memory used
11179816	11:17:35 31 Mar 2026	hduads2026_24320124	1401_Gamers	Visual C++ 2022	Accepted		0.203	1 264 KB

2025:

Recent submissions

Author: [hduads2026_24320124](#) • Problem: [Line Fighting](#)

ID	Date	Author	Problem	Language	Judgement result	Test #	Execution time	Memory used
11183939	18:22:33 7 Apr 2026	hduads2026_24320124	2025_Line Fighting	Visual C++ 2022	Accepted		0.015	460 KB

1207:

Recent submissions

Author: [hduads2026_24320124](#) • Problem: [Median on the Plane](#)

ID	Date	Author	Problem	Language	Judgement result	Test #	Execution time	Memory used
11179797	10:09:06 31 Mar 2026	hduads2026_24320124	1207_Median on the Plane	Visual C++ 2022	Accepted		0.046	704 KB

Explanation of algorithm:

1401:

This problem uses **divide-and-conquer** to solve the defective chessboard tromino tiling problem:

1. Divide the $2^n \times 2^n$ board into four quadrants of size $2^{n-1} \times 2^{n-1}$.
2. Identify which quadrant contains the missing cell.
3. Place one L-shaped tromino at the center, covering one center cell from each of the other three quadrants, so that every quadrant now has exactly one missing cell.

4. Recursively apply the same process to all four quadrants.
5. Assign a unique number to each tromino placed; all three cells of the same tromino receive the same number.

2025:

Total fights = total pairs – pairs inside same team.

To maximize fights, minimize inside-team pairs.

For fixed team count m , inside pairs are minimized when team sizes are as equal as possible.

Let $a = \lfloor n/m \rfloor$, $r = n \bmod m$:

- r teams have size $a + 1$
- $m - r$ teams have size a

$$\text{Inside pairs} = (m - r) \cdot \binom{a}{2} + r \cdot \binom{a+1}{2}$$

Fights for given m :

$$\text{fights}(m) = \binom{n}{2} - \text{inside_pairs}$$

Try all m from 2 to k , take maximum.

1207:

First, find the bottom left point as the reference point O , and then sort the other points by their polar angle relative to O . Next, use double pointer scanning: fixed pointer i points to the currently examined point P_i , pointer j points to the last point that satisfies $\angle P_i O P_j < 180^\circ$. Due to the ordered polar angle, j monotonically increases with i . For the line OP_i , the points between $i+1$ and j are located on the left side of the line, while the remaining points are located on the right side (excluding O and P_i). When the number of points on the left or right side is equal to $N/2 - 1$, and the reference point O is added, there are exactly $N/2$ points on that side. At this time, the output O and P_i numbers are obtained.

Computational complexity of algorithm:

1401:

Time Complexity: $O(4^n)$

Recurrence relation: $T(n) = 4T(n-1) + O(1)$, which solves to $T(n) = O(4^n)$.

Space Complexity: $O(4^n)$

The board stores $2^n \times 2^n = 4^n$ integers. Recursion stack depth is $O(n)$, which is negligible.

2025:

- Per test case: $O(k)$ time, $O(1)$ space
- $k \leq n \leq 10^4$, $T \leq 10$
- Total: $O(T \cdot k) \leq 10^5$ operations, fast enough.

1207:

Time complexity:

Find benchmark: $O(N)$

Polar sorting: $O(N \log N)$ (using quicksort or merge sort)

Double pointer scan: $O(N)$ (j pointer monotonically increases, moving no more than N times in total)

Total time complexity: $O(N \log N)$

Space complexity:

Store the coordinates and numbers of N points: $O(N)$

Extra space used for sorting: $O(\log N)$ (recursive stack) or $O(N)$ (merge sort)

Total space complexity: $O(N)$

Source code:

1401:

```
#include <iostream>
```

```
#include <vector>
```

```

using namespace std;

vector<vector<int>>> board;

int tile = 1;

void place(int x, int y, int tx, int ty, int sx, int sy) {
}

void tromino(int tx, int ty, int x, int y, int size) {
    if (size == 1) return;

    int t = tile++;

    int s = size / 2;

    if (x < tx + s && y < ty + s) {
        tromino(tx, ty, x, y, s);
    } else {
        board[tx + s - 1][ty + s - 1] = t;
        tromino(tx, ty, tx + s - 1, ty + s - 1, s);
    }

    if (x < tx + s && y >= ty + s) {
        tromino(tx, ty + s, x, y, s);
    } else {
        board[tx + s - 1][ty + s] = t;
        tromino(tx, ty + s, tx + s - 1, ty + s, s);
    }

    if (x >= tx + s && y < ty + s) {
        tromino(tx + s, ty, x, y, s);
    } else {
        board[tx + s][ty + s - 1] = t;

```

```

        tromino(tx + s, ty, tx + s, ty + s - 1, s);
    }

    if (x >= tx + s && y >= ty + s) {

        tromino(tx + s, ty + s, x, y, s);

    } else {

        board[tx + s][ty + s] = t;

        tromino(tx + s, ty + s, tx + s, ty + s, s);

    }

}

int main() {

    int n, x, y;

    cin >> n >> x >> y;

    int size = 1 << n;

    board.assign(size, vector<int>(size, 0));

    x--; y--;

    board[x][y] = 0;

    tromino(0, 0, x, y, size);

    for (int i = 0; i < size; i++) {

        for (int j = 0; j < size; j++) {

            cout << board[i][j] << (j == size - 1 ? "\n" : " ");

        }

    }

    return 0;

}

```

2025:

```
#include <iostream>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
long long fights(int n, int m) {
```

```
    long long total_pairs = (long long)n * (n - 1) / 2;
```

```
    int a = n / m;
```

```
    int r = n % m;
```

```
    long long inner_pairs = (long long)(m - r) * a * (a - 1) / 2  
                           + (long long)r * (a + 1) * a / 2;
```

```
    return total_pairs - inner_pairs;
```

```
}
```

```
int main() {
```

```
    int T;
```

```
    cin >> T;
```

```
    while (T--) {
```

```
        int n, k;
```

```
        cin >> n >> k;
```

```
        long long ans = 0;
```

```
        // m 从 2 到 k, 也可以到 n (因为  $k \leq n$ )
```

```
        for (int m = 2; m <= k; ++m) {
```

```
            ans = max(ans, fights(n, m));
```

```
        }
```

```
        cout << ans << "\n";
```

```
}
```

```
        return 0;
    }
```

1207:

```
#include <iostream>

#include <vector>

#include <algorithm>

#include <cmath>
```

```
using namespace std;
```

```
struct Point {
    long long x, y;
    int id;
};
```

```
long long cross(const Point& a, const Point& b, const Point& p) {
    return (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x);
}
```

```
int main() {
    int N;

    cin >> N;

    vector<Point> points(N);

    for (int i = 0; i < N; i++) {
        cin >> points[i].x >> points[i].y;
```

```
    points[i].id = i + 1;
}
```

```
// 找到最左下角的点
```

```
int base_idx = 0;
for (int i = 1; i < N; i++) {
    if (points[i].y < points[base_idx].y ||
        (points[i].y == points[base_idx].y && points[i].x < points[base_idx].x)) {
        base_idx = i;
    }
}
```

```
swap(points[0], points[base_idx]);
```

```
Point base = points[0];
```

```
// 按极角排序
```

```
sort(points.begin() + 1, points.end(), [&base](const Point& a, const Point& b) {
    long long cross_val = (a.x - base.x) * (b.y - base.y) - (a.y - base.y) * (b.x -
base.x);
    if (cross_val != 0) return cross_val > 0;
    long long dist_a = (a.x - base.x) * (a.x - base.x) + (a.y - base.y) * (a.y - base.y);
    long long dist_b = (b.x - base.x) * (b.x - base.x) + (b.y - base.y) * (b.y - base.y);
    return dist_a < dist_b;
});
```

```
// 双指针找答案
```

```
int j = 1;
```



```

for (int i = 1; i < N; i++) {
    if (j <= i) j = i + 1;

    // 找到与 points[i] 相对的点
    while (j < N && cross(base, points[i], points[j]) > 0) {
        j++;
    }

    int left_count = j - i - 1;
    int right_count = (N - 2) - left_count;

    if (left_count == N/2 - 1 || right_count == N/2 - 1) {
        cout << base.id << " " << points[i].id << endl;
        return 0;
    }
}

return 0;
}

```